

REPORT CONTENT SHEET

Raw material for your futuristic report — Build It in English

Hoja de contenido: materia prima del informe futurista

Student / Estudiante	Moises Isai Cornelio Hernandez
Project / Proyecto	FIELD KIT — Technician Service Log
Pitch	A mobile companion for maintenance technicians that logs visits, tracks tasks and generates a client-ready summary in seconds.
Environment / Entorno	Python 3.x + Flet, developed in VS Code, tested on Android
Report medium / Medio del informe	_____
Date / Fecha	2026-09-03
Teacher / Docente	Ing. Javier Arturo García Mineros

Reminder: this sheet is not the report. Transform this content into your futuristic PDF, following the design brief.

Esta hoja no es el informe: transforme este contenido en su PDF futurista siguiendo el brief de diseño.

2. The environment used

This project was built with Python 3 and Flet, a framework that lets a single Python codebase run as a native-feeling app on both desktop and Android. VS Code was used as the IDE, with the Flet CLI to run and package the app. Flet fits Field Kit well because the app is a short list of simple, form-based screens (visits, tasks, a summary) — exactly the kind of UI Flet's built-in controls (Column, ListView, Card) are designed for, without needing a heavier native Android toolchain.

Este proyecto se construyó con Python 3 y Flet, un framework que permite que un mismo código en Python funcione como app nativa en escritorio y Android. Se usó VS Code como IDE. Flet encaja bien con Field Kit porque la app son pantallas simples basadas en listas y formularios.

2.1 Alternative not used

An alternative environment would have been Kivy, another Python framework for cross-platform mobile apps. Kivy offers more control over custom UI and animations, but it has a steeper learning curve, a less modern-looking default style, and a more complex Android packaging process than Flet's. For a form-and-list app like Field Kit, Flet's simplicity and faster build-to-device cycle made it the better fit.

Una alternativa habría sido Kivy, otro framework de Python multiplataforma. Kivy da más control sobre UI personalizada, pero tiene una curva de aprendizaje más pronunciada y un empaquetado a Android más complejo. Para una app de listas y formularios como Field Kit, Flet fue la mejor opción.

3. Tools

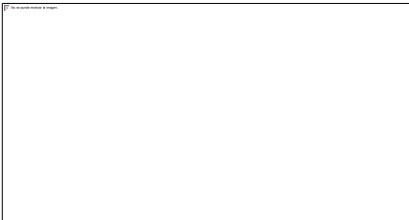
TOOL	VERSION	WHAT IT WAS USED FOR (ENGLISH)
Visual Studio 202	18.9.2	Main IDE used to build, edit, run, and debug the .NET MAUI application (KitDeCampo).
.NET MAUI	8.0	Cross-platform framework providing native UI controls (pages, borders, lists, inputs) and layout structure for the mobile app.
C#	12.0	Primary programming language used to write business logic, task handling, time calculation, and data persistence services.
NuGet / MSBuild	8.0	Package management and build engine used to compile dependencies and target Android/Windows platforms.
Git	2.44+	Version control system used to track changes, manage commits, and sync progress to the GitHub repository (FieldKit-).

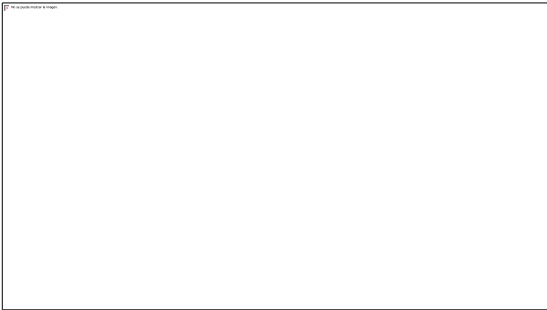
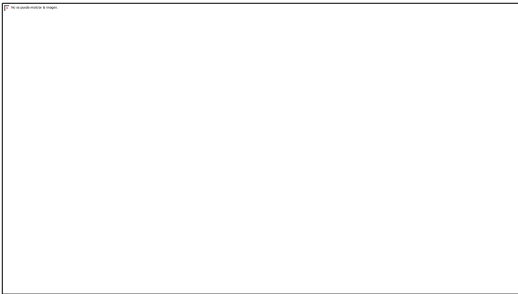
4. Resources

RESOURCE TYPE	DETAIL	CONTRIBUTION TO THE PROJECT
Hardware	ProcesadorAMD Ryzen 3 7320U with Radeon Graphics (2.40 GHz) RAM instalada8.00 GB (7.24 GB utilizable)	run and test the app on a real device

	Tarjeta gráfica AMD Radeon(TM) Graphics (486 MB) Tipo de sistema Sistema operativo de 64 bits, procesador x64	
Time invested	45 min × 6 blocks over 3 days	Time spent building, validating, persisting and documenting the app
Documentation / tutorials	Flet official documentation .docs	Used to learn the Column, Row, Card and ListView controls and how to build the app for Android
Libraries or packages	flet	Provided all UI controls and the runtime that lets one Python codebase run on desktop and Android
AI or assistance used	Claude (Anthropic AI assistant)	[fill in: describe exactly what you used it for — e.g. organising this report content, or a specific coding question you asked]

5. Functionality

FEATURE	WHAT THE USER DOES AND SEES (ENGLISH)	EVIDENCE
Create a visit	The user taps "+ New Visit / Nueva Visita", enters the client name and date, and saves. The new visit appears at the top of the main visit list.	
Add tasks to a visit	Inside an open visit, the user enters a task description and duration in minutes, then taps "Add / +" to attach	

	the task to that visit.	
Mark tasks as done	The user checks the checkbox next to a task upon completion, updating the checklist item instantly.	
Automatic visit status	The visit status updates automatically between Open (Abierta), In Progress (En progreso), or Closed (Cerrada) depending on task completion, without manual user input.	
Total time per visit	The app sums the duration of all added tasks and displays the total accumulated time (e.g., Total: 45 min) in real time on the visit head	

5.1 Out of scope

This version of Field Kit does not persist visits between sessions (there is no local JSON file yet, so data resets each time the app restarts), does not include a per-task timer, and does not copy the summary to the clipboard automatically. These three were intentionally left as stretch goals so the MUST features could be finished and tested first.

Esta versión de Field Kit no guarda las visitas entre sesiones (todavía no hay archivo JSON local, así que los datos se pierden al reiniciar la app), no incluye un cronómetro por tarea y no copia el resumen al portapapeles automáticamente. Se dejaron fuera a propósito para terminar y probar primero las funciones obligatorias.

6. Architecture and data model

[fill in / adjust to match your real folder structure] The project is organised around two data classes, Visit and Task, kept in a models module. The user interface is split into one Flet view per screen — the visit list, the new-visit form, the task checklist and the summary view — each responsible for a single job. The main app file wires the views together and holds the in-memory list of visits that is passed down as the user navigates.

[ajuste a su estructura real] El proyecto se organiza en dos clases de datos, Visit y Task, en un módulo de modelos. La interfaz se divide en una vista de Flet por pantalla, cada una responsable de una sola tarea.

6.1 Data persistence

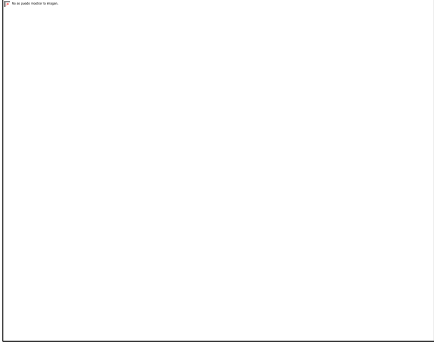
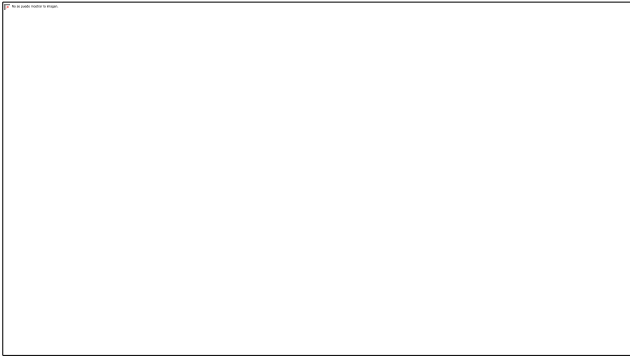
At this stage, Field Kit keeps all visits and tasks in memory only, inside a Python list that lives for as long as the app is running; closing the app discards everything. The next step — required before this can be considered finished — is to read a local JSON file on startup (creating an empty one if it does not exist) and write it after every change, with error handling for a missing, unreadable or corrupted file.

En esta etapa, Field Kit mantiene las visitas y tareas solo en memoria; al cerrar la app se pierden los datos. El siguiente paso, necesario para considerarla terminada, es leer un archivo JSON local al iniciar y guardarlo después de cada cambio, manejando errores si el archivo no existe o está dañado.

Insert here the diagram of your data model.
Inserte aquí el diagrama de su modelo de datos.

7. Code highlights

FILE AND METHOD	WHAT IT DOES	WHY IT IS INTERESTING
	VisitsPage is a screen in the Kit de Campo app that allows users to view and manage service visits. It includes a bilingual header, a list of visits showing details such as the client, date, duration, and status, and a button to create new visits. If there are no saved visits, it displays a message indicating	The IDE and XAML code show that the Kit de Campo app is being developed with .NET MAUI for an Android tablet. The interface is designed for field technicians, using large touch targets, cards, and clear spacing. It uses data binding and modern MAUI layouts such as Grid and VerticalStackLayout, while a custom burgundy and gold color scheme provides a strong visual hierarchy and readability. The project also appears to contain a naming inconsistency between VisitsPage and SummaryPage, likely caused by an unfinished refactoring or copy-paste artifact.

	that there are no visits yet.	
	TasksPage is a screen in the KitDeCampo app designed to help field technicians manage tasks during a service visit. It displays customer and visit information, a list of tasks with descriptions, completion checkboxes, and estimated durations. Users can add new tasks by entering a description and the number of minutes required. At the bottom, a button allows them to view a summary of the visit and its completed tasks.	The TasksPage uses a practical and interactive design that lets field technicians manage tasks directly from one screen. It keeps important visit information visible, allows users to add and complete tasks without changing screens, and combines data binding with event handlers for efficient interaction. The consistent maroon and gold styling, rounded corners, and thin borders also make the interface clear and accessible on touchscreen devices.
	NewVisitPage is the backend logic for creating a new service visit in the KitDeCampo app. It sets the visit date, validates the customer name, automatically generates a unique ID, and creates the visit with an empty	This code is interesting because it shows how a complete visit-creation process can be handled automatically. It validates the customer information, generates a unique visit ID, saves the data using DataService, and immediately navigates the technician to the task page. This creates a simple and efficient workflow with minimal manual input.

	task list. The new visit is then saved through the DataService, and the user is taken directly to TasksPage to begin adding tasks.	
--	--	--

Paste the three code excerpts below, with their English comments.

Pegue los tres fragmentos de código con sus comentarios en inglés.

8. Test log

ID	WHAT WAS TESTED	INPUT	EXPECTED RESULT	ACTUAL RESULT	STATUS
T-01	Create visit with empty client name	Client name = ""	Message asking for a name; no visit created	As expected	PASS
T-02	Create a valid visit	Client = "Acme Corp", today's date	Visit appears at the top of the visit list	As expected	PASS
T-03	Add a task to a visit	Task description = "Replace filter"	Task appears in the checklist	As expected	PASS
T-04	Mark all tasks as done	All tasks checked	Status changes automatically to "Closed"	As expected	PASS
T-05	Generate summary with zero tasks	Visit with no tasks	Summary still generates, states no tasks were logged	As expected	PASS
T-06	Very long client name (200 characters)	200-character string	Layout does not break	As expected	PASS
T-07	Delete a visit with nothing selected	No visit selected, delete pressed	Handled politely, no crash	As expected	PASS

9. Challenges and conclusions

PROBLEM	WHAT I SEARCHED IN ENGLISH	HOW I SOLVED IT
The app crashed when the client name field was left empty and the form tried to save the visit	"flet textfield empty value crash"	Added a validation check before saving that shows an error message and stops the save if the field is blank
The visit list did not refresh automatically after adding a new visit	"flet listview not updating ui"	Learned that Flet needs <code>page.update()</code> (or <code>.update()</code> on the control) after changing the data, and added it after every state change
Building the app for Android failed the first time	"flet build apk error"	Updated the Flet CLI to the latest version and re-ran the build, which fixed the missing dependency error

9.1 Conclusions

Building Field Kit taught me that a mobile app looks simple from the user's side but needs careful planning underneath — especially the data model and the validation rules, which I ended up adjusting more than once as new edge cases appeared. If I started this project again, I would design the Visit and Task classes on paper before writing any interface code, instead of changing them as I went. Working with Python and Flet also showed me how fast it is to go from an idea to a working screen, which I did not expect from a framework I had never used before. With this same environment, I would like to build a small app that connects to a real API instead of only working with local data, and add local JSON persistence from the very first version instead of leaving it for later.

Construir Field Kit me enseñó que una app móvil se ve simple desde el lado del usuario, pero necesita mucha planificación detrás, especialmente en el modelo de datos y las validaciones, que tuve que ajustar más de una vez. Si empezara de nuevo, diseñaría las clases Visit y Task en papel antes de programar la interfaz. Trabajar con Python y Flet también me mostró qué tan rápido se puede pasar de una idea a una pantalla funcionando. Con este mismo entorno, me gustaría construir una app que se conecte a una API real y agregar la persistencia en JSON desde el inicio, en vez de dejarla para después.

Declaration / Declaración

I declare that I wrote the code and the report myself, that every identifier and comment is in English, and that I can explain any part of this work on request.

Declaro que escribí el código y el informe, que todo identificador y comentario está en inglés y que puedo explicar cualquier parte de este trabajo.

Student / Estudiante

Ing. Javier A. García Mineros
Teacher / Docente